

Clase 4 – JavaScript DOM & TypeScript Basics

IIC2182 – Interfaces y Experiencia de Usuario



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Primer Semestre 2026

Parte I

JavaScript & el DOM

Parte I

JavaScript & el DOM

Seleccionar elementos — `querySelector`

`querySelector` — primer match

```
1 // Por tag
2 const title = document.querySelector('h1');
3
4 // Por clase
5 const card = document.querySelector('.product-card');
6
7 // Por ID
8 const app = document.querySelector('#app');
9
10 // Selector compuesto (como CSS)
11 const price = document
12   .querySelector('.product-card .price');
13
14 // ⚠ Retorna null si no encuentra nada
15 const ghost = document.querySelector('.no-existe');
16 console.log(ghost); // null
```

`querySelectorAll` — todos los matches

```
1 // Retorna un NodeList (parecido a un array)
2 const links = document.querySelectorAll('nav a');
3 console.log(links.length); // 2
4
5 // Iterar sobre los resultados
6 links.forEach(link => {
7   console.log(link.textContent);
8 });
9
10 // Convertir a array para usar .map(), .filter()
11 const hrefs = [...links].map(a => a.href);
12 console.log(hrefs);
```



`querySelector` usa la misma sintaxis que CSS. Si sabés CSS, ya sabés seleccionar elementos del DOM.

Modificar contenido y atributos

Contenido de texto

```
1  const title = document.querySelector('h1');
2
3  // textContent - texto plano (seguro)
4  title.textContent = 'Bienvenido';
5
6  // innerHTML - puede incluir HTML
7  // ⚠ Cuidado con XSS si el dato viene del usuario
8  title.innerHTML = 'Oferta <em>especial</em>';
9
10 // Leer contenido
11 console.log(title.textContent); // "Oferta especial"
```

textContent vs innerHTML

textContent es seguro — escapa HTML automáticamente.
innerHTML interpreta HTML — nunca usarlo con datos del usuario sin sanitizar.

Atributos HTML

```
1  const img = document.querySelector('img');
2
3  // Leer y escribir atributos
4  img.src = '/images/product.jpg';
5  img.alt = 'Foto del producto';
6
7  // Atributo personalizado - data-*
8  const card = document.querySelector('.card');
9  card.dataset.productId = '42';
10 // → <div class="card" data-product-id="42">
11 console.log(card.dataset.productId); // "42"
```

data-* attributes

Permiten guardar **datos custom** en elementos HTML. Muy útil para pasar IDs, estados, o config a event handlers.

Clases CSS y estilos dinámicos

classList — la forma correcta

```
1  const btn = document.querySelector('.btn');
2
3  // Agregar y quitar clases
4  btn.classList.add('active');
5  btn.classList.remove('disabled');
6
7  // Toggle — agrega si no está, quita si está
8  btn.classList.toggle('loading');
9
10 // Verificar si tiene una clase
11 if (btn.classList.contains('active')) {
12   console.log('Botón activo');
13 }
14
15 // Agregar múltiples clases de una vez
16 btn.classList.add('btn-primary', 'rounded', 'shadow')
```

style — solo para valores dinámicos

```
1  const el = document.querySelector('.box');
2
3  // Modificar estilos directamente
4  el.style.backgroundColor = '#3b82f6';
5  el.style.padding = '16px';
6  el.style.borderRadius = '8px';
7
8  // Leer estilos computados
9  const computed = getComputedStyle(el);
10 console.log(computed.backgroundColor);
```



Regla de oro

classList para estados o clases predefinidos (active, disabled, hidden) → reusable y mantenible. **style** solo para valores calculados en runtime (posiciones, tamaños dinámicos).

Crear y eliminar elementos

Crear e insertar

```
1 // 1. Crear el elemento
2 const card = document.createElement('div');
3 card.className = 'product-card';
4
5 // 2. Agregar contenido
6 card.innerHTML = `
7   <h2>Teclado Mecánico</h2>
8   <span class="price">$29.990</span>
9   <button class="add-to-cart">Agregar</button>
10 `;
11
12 // 3. Insertar en el DOM
13 const container = document.querySelector('#products');
14 container.appendChild(card);
15
16 // insertAdjacentHTML - más flexible
17 // (beforeBegin, beforeend, afterbegin, afterend)
18 container.insertAdjacentHTML('beforeend', `
19   <div class="product-card">
20     <h2>Mouse</h2>
21     <span class="price">$14.990</span>
22   </div>
23 `);
```

Eliminar elementos

```
1 // Eliminar un elemento
2 const oldCard = document.querySelector('.outdated');
3 oldCard.remove();
4
5 // Vaciar un contenedor
6 const list = document.querySelector('#product-list');
7 list.innerHTML = '';
8
9 // Reemplazar contenido completo
10 list.innerHTML = '<p>No hay productos</p>';
```

</> Template literals + innerHTML

Combinar **template literals** con **innerHTML** es el patrón más común para renderizar datos en el DOM sin frameworks.

Event Listeners — Reaccionar a la interacción

addEventListener — escuchar eventos

```
1  const btn = document.querySelector('#add-btn');
2
3  // Click
4  btn.addEventListener('click', () => {
5    console.log('¡Botón clickeado!');
6  });
7
8  // Con el objeto event
9  btn.addEventListener('click', (event) => {
10    console.log('Target:', event.target);
11    console.log('Tipo:', event.type); // "click"
12  });
13
14  // Input — cambios en tiempo real
15  const search = document.querySelector('#search');
16  search.addEventListener('input', (e) => {
17    console.log('Buscar:', e.target.value);
18  });
```

Eventos comunes en UI

```
1  // Submit de formulario
2  const form = document.querySelector('form');
3  form.addEventListener('submit', (e) => {
4    e.preventDefault(); // ← evitar recarga
5    const data = new FormData(form);
6    console.log('Nombre:', data.get('name'));
7  });
8
9  // Keydown — atajos de teclado
10 document.addEventListener('keydown', (e) => {
11   if (e.key === 'Escape') {
12     closeModal();
13   }
14 });
15
16 // Change — selects, checkboxes
17 const select = document.querySelector('#sort');
18 select.addEventListener('change', (e) => {
19   sortProducts(e.target.value);
20 });
```


Renderizar listas desde datos

El patrón más importante: datos → HTML

```
1  const products = [  
2    { id: 1, name: 'Teclado Mecánico', price: 29990, stock: 5 },  
3    { id: 2, name: 'Monitor 4K',          price: 89990, stock: 0 },  
4    { id: 3, name: 'Mouse Inalámbrico', price: 14990, stock: 12 },  
5  ];  
6  
7  function renderProducts(products) {  
8    const container = document.querySelector('#product-list');  
9    container.innerHTML = products  
10     .filter(p => p.stock > 0)  
11     .map(p => `  
12       <div class="product-card" data-id="${p.id}">  
13         <h3>${p.name}</h3>  
14         <span class="price">${p.price.toLocaleString('es-CL')}</span>  
15         <button class="delete-btn" data-id="${p.id}">Eliminar</button>  
16       </div>  
17     `).join('');  
18  }  
19  
20  renderProducts(products);
```

Event Delegation

Un solo listener para manejar muchos elementos — incluso los futuros

El problema

```
1 // ❌ Un listener por botón
2 document.querySelectorAll('.delete-btn')
3   .forEach(btn => {
4     btn.addEventListener('click', () => {
5       deleteProduct(btn.dataset.id);
6     });
7   });
8 // Elementos creados después NO tienen listener
```

Los eventos **burbujean** desde el elemento clickeado hacia arriba en el DOM. Podemos escuchar en un ancestro común.

La solución — delegation

```
1 // ✅ Un listener en el contenedor
2 const list = document
3   .querySelector('#product-list');
4
5 list.addEventListener('click', (e) => {
6   const btn = e.target.closest('.delete-btn');
7   if (!btn) return;
8
9   const id = btn.dataset.id;
10  deleteProduct(id);
11 });
12 // Funciona para elementos presentes Y futuros
```



Cuándo usar delegation

Listas, tablas, menús — cualquier UI con elementos **dinámicos**. Es el patrón que usan React y Vue internamente.

Destructuring & Spread — Esenciales para frontend

Destructuring — extraer valores

```
const product = { name: 'Teclado', price: 29990, stock: 5 };
const { name, price } = product;
console.log(name, price);

// Arrays — patrón useState de React
const [first, ...rest] = [10, 20, 30, 40];
console.log(first, rest);
```

Teclado, 29990
10, [20, 30, 40]

Spread — copiar y combinar

```
const product = { name: 'Teclado', price: 29990 };

// Actualizar sin mutar (patrón React/Vue)
const updated = { ...product, price: 24990 };
console.log('Original:', product.price);
console.log('Updated:', updated.price);

// Arrays: agregar sin mutar
const items = ['a', 'b'];
console.log([...items, 'c']);
```

Original:, 29990
Updated:, 24990
["a", "b", "c"]



Destructuring y spread son los patrones más usados en **React**: props, state updates, event handlers.

Ejercicio 1

Manipulando la Tienda IIC2182

Ejercicio 1 — Manipulando la Tienda IIC2182

Abre la demo interactiva y escribe JavaScript en el editor para completar cada tarea

Contexto — la demo ya tiene:

```
1 // Array global con productos
2 products // [{ id, name, price, stock, category
3
4 // Funciones disponibles
5 renderProducts() // re-renderiza la lista
6 // Estructura del DOM
7 // #page-title, #search, #sort-select
8 // #product-list > .product-card[data-id]
9 // #style-demo-box
```

Tareas — escribe en el editor de la demo:

1. Usa `querySelector` para cambiar el titulo de la tienda a tu nombre
2. Usa `querySelectorAll` para contar cuántas product cards hay visibles
3. Agrega la clase `active` al demo box con `classList`
4. Usa `style` para cambiar el `backgroundColor` del demo box a un color custom
5. Crea un nuevo producto con `products.push( ... )` y llama `renderProducts()`
6. Bonus: Usa `querySelectorAll + forEach` para loggear el nombre de cada producto visible

Ejercicio 1 — Solución

```
1 // 1. Cambiar titulo
2 document.querySelector('#page-title').textContent = 'Mi Tienda';
3
4 // 2. Contar cards visibles
5 const cards = document.querySelectorAll('.product-card');
6 console.log('Productos visibles:', cards.length);
7
8 // 3. classList - toggle active
9 document.querySelector('#style-demo-box').classList.add('active');
10
11 // 4. style - color custom
12 document.querySelector('#style-demo-box').style.backgroundColor = '#8b5cf6';
13
14 // 5. Agregar producto y re-renderizar
15 products.push({ id: 99, name: 'Parlante BT', price: 19990, stock: 7, category: 'audio' });
16 renderProducts();
17
18 // 6. Bonus - loggear nombre de cada producto
19 document.querySelectorAll('.product-card .name').forEach(el => console.log(el.textContent));
```


Parte II

TypeScript Basics

Definition — ¿Qué es TypeScript?

TypeScript es un superset de JavaScript

```
1      TypeScript
2
3      JavaScript ✓
4      + tipos    ✓
5      + interfaces ✓
6      + generics  ✓
7
```

Cómo funciona: compila a JS puro

```
1      archivo.ts
2      ↓
3      TypeScript Compiler (tsc)
4      ↓
5      ✅ Verifica tipos
6      ❌ Reporta errores → antes de ejecutar
7      ↓
8      archivo.js ← JavaScript puro
```

Cualquier `.js` válido es `.ts` válido.

```
1  // JavaScript - válido como TypeScript
2  const greet = (name) => `Hola, ${name}`;
3
4  // TypeScript - con anotaciones de tipos
5  const greet2 = (name: string): string =>
6    `Hola, ${name}`;
```

Los tipos no existen en runtime. Son solo una herramienta para el desarrollo. El output es JS estándar.

Errores en build-time, no en producción

Sin TypeScript — falla en runtime

```
1 function getUser(id) {  
2   return users[id].nmae; // typo silencioso  
3 }  
4 // Se ejecuta sin error...  
5 // El usuario ve "undefined" en la pantalla 🤖
```



Dato de industria

Según un estudio de Airbnb, después de migrar a TypeScript, **el 38% de los bugs que llegaban a producción** habrían sido detectados por el type checker.

Fuente: Airbnb Engineering, "ts-migrate" (2020)

Con TypeScript — falla en el editor

```
1 function getUser(id: number) {  
2   return users[id].nmae;  
3   // ✖ Property 'nmae' does not exist  
4   //   on type User. Did you mean 'name'?  
5 }  
6 // → Error ANTES de ejecutar
```



Adopción masiva

TypeScript es el **4to lenguaje más usado** en GitHub (2024). El 78% de los repos JavaScript populares ya usan TypeScript.

Autocomplete — Tu IDE se vuelve tu documentación

```
interface Product {  
  id: string;  
  name: string;  
  price: number;  
  stock: number;  
  category: string;  
}  
  
const p: Product = {  
  id: '1',  
  name: 'Teclado',  
  price: 29990,  
  // IDE sugiere: stock, category  
  // y avisa si te falta un campo ✖  
};  
  
// Al escribir "p." el IDE muestra:  
// → p.id, p.name, p.price, p.stock...
```



Documentación

Con TypeScript, **no necesitas memorizar** la forma de cada objeto. El IDE te muestra los campos disponibles, sus tipos, y documenta la API mientras escribes.



Ejemplo: APIs externas

Librerías como **React**, **Next.js**, **Express** vienen con tipos incluidos. Al escribir un componente, el IDE te sugiere todas las props disponibles sin abrir la documentación.

Paquetes `@types/*` en npm para librerías sin tipos nativos

Refactor seguro — Cambiar sin miedo

```
// Renombrás `price` a `amount` en la interface
interface Product {
  id: string;
  name: string;
  amount: number; // ← antes era `price`
}

const products: Product[] = [
  { id: '1', name: 'Teclado', amount: 29990 },
  { id: '2', name: 'Mouse', amount: 14990 },
];

// TS marca TODOS los usos en el proyecto:
const total = products
  .reduce((s, p) => s + p.price, 0);
//                               ↑ X
// Property 'price' does not
// exist on type Product

// 📍 El IDE encuentra y reemplaza
// de forma segura en TODO el codebase
```



Caso real: Stripe

Stripe migró su dashboard (millones de líneas) a TypeScript. Reportan que **los refactors que antes tomaban semanas** ahora se hacen en horas — el compilador encuentra todos los puntos de cambio.



Sin tipos: refactor = ruleta rusa

En JS puro, renombrar un campo requiere buscar con "Find & Replace" y **rezar que no se escape ningún uso**. Los tests pueden no cubrir todos los paths.

Especialmente peligroso con propiedades dinámicas y strings

Cuando falla el tipo: coerción silenciosa

JavaScript tiene coerción de tipos implícita — y es silenciosa

```
// Operaciones que deberían fallar... y no lo hacen ▶  
console.log(5 + "3");      // → "53" 🤯 (concatena, no su  
console.log("5" - 3);      // → 2      (coerción distinta)  
console.log(true + true);  // → 2  
console.log([] + []);      // → ""  
console.log([] + {});      // → "[object Object]"
```

53

2

2

[object Object]

Un bug de UI real:

```
1 // El input HTML siempre retorna string  
2 const price = "29990"; // de un <input>  
3 const qty   = "3";     // de un <input>  
4 console.log(price * qty); // → 89970 (suerte)  
5 console.log(price + qty); // → "299903" 🤯
```

Con TypeScript:

```
function calcTotal(price: number, qty: number) { ▶  
  return price * qty;  
}  
calcTotal("29990", "3");  
// ✖ Argument of type 'string' is not  
//   assignable to parameter of type 'number'  
calcTotal(Number("29990"), Number("3")); // ✔
```


Cuando falla el tipo: el contrato con la API se rompe

El backend cambia y el frontend se rompe en silencio

Sin TypeScript

```
1 // Semana 1 - backend retorna { full_name: "Ana" }
2 function ProfileHeader({ user }) {
3   return `<h1>${user.full_name}</h1>`; // ✅
4 }
5
6 // Semana 3 - backend renombra a { name: "Ana" }
7 // Frontend muestra: undefined 🤖
8 // Sin error en consola
```

Con TypeScript

```
interface ApiUser {
  id: number;
  name: string;    // ← actualizado
  email: string;
}

function ProfileHeader({ user }: { user: ApiUser }) {
  return `<h1>${user.full_name}</h1>`;
  // ❌ Property 'full_name' does not
  //    exist on type ApiUser
}
```

Error en **build-time**, no una pantalla en blanco en producción.

Cuando falla el tipo: null y undefined

El error de runtime más frecuente — "Cannot read properties of undefined"

Sin TypeScript

```
1  const user = fetchUser();
2  // podría retornar null si no hay sesión
3
4  console.log(user.name);
5  // ✨ TypeError: Cannot read
6  //   properties of null
7
8  function getAvatar(user) {
9    return user.profile.avatar.url;
10   // ← tres niveles de "podría ser null"
11 }
12 // Si CUALQUIERA es null → crash
```

Con TypeScript

```
interface User {
  name: string;
  profile?: {           // puede ser undefined
    avatar?: {         // puede ser undefined
      url: string;
    };
  };
}

function getAvatar(user: User | null) {
  return user?.profile?.avatar?.url;
  // optional chaining – safe access ✅
}

function getDisplayName(user: User | null) {
  if (!user) return 'Invitado';
  // ← TS exige manejar el caso null
  return user.name; // TS sabe que no es null
}
```

Type Annotations — Primitivos y Arrays

```
// — Primitivos —————  
const productName: string = 'Teclado Mecánico';  
const price: number       = 29990;  
const isAvailable: boolean = true;  
  
// — Arrays —————  
const tags: string[]      = ['oferta', 'nuevo', 'destacado'];  
const prices: Array<number> = [9990, 19990, 29990];  
  
// — Union types — puede ser uno de varios valores —————  
type Status = 'idle' | 'loading' | 'success' | 'error';  
type Variant = 'primary' | 'secondary' | 'ghost' | 'danger';  
  
let currentStatus: Status = 'idle';  
currentStatus = 'loading'; // ✅  
currentStatus = 'waiting'; // ❌ not assignable to type Status
```

Los Union types son perfectos para representar estados de UI: loading, error, success...

Type Annotations — Funciones

```
1  // — Anotar parámetros y retorno —————
2  function formatPrice(amount: number, locale: string = 'es-CL'): string {
3      return new Intl.NumberFormat(locale, { style: 'currency', currency: 'CLP' }).format(amount);
4  }
5
6  const double = (n: number): number ⇒ n * 2;
7
8  // void — cuando la función no retorna un valor útil
9  const logError = (msg: string): void ⇒ console.error(`[error] ${msg}`);
10
11 // — Parámetro opcional con ? —————
12 function greet(name: string, title?: string): string {
13     return title ? `${title} ${name}` : name;
14 }
```

Type inference — TS infiere automáticamente

```
1  const name    = 'Ana';    // TS infiere: string
2  const age     = 25;       // TS infiere: number
3  const active  = true;     // TS infiere: boolean
4  const prices  = [10, 20]; // TS infiere: number[]
5
6  // Anotar solo cuando TS no puede inferir
7  let status: Status = 'idle'; // necesario — TS inferiría string
```

Interfaces & Types

`interface` — la forma de un objeto

```
1 interface Product {
2   id: string;
3   name: string;
4   price: number;
5   stock: number;
6   category: string;
7   imageUrl?: string; // opcional
8   readonly createdAt: Date; // immutable
9 }
10
11 // Extender — herencia de tipos
12 interface FeaturedProduct extends Product {
13   badge: string;
14   discountPercent: number;
15 }
```

`type` — más flexible

```
1 // Alias para cualquier tipo
2 type ID = string | number;
3 type Nullable<T> = T | null;
4
5 // Union de objetos con "forma"
6 type ApiResponse =
7   | { status: 'success'; data: Product[] }
8   | { status: 'error'; message: string };
9
10 // Intersection — combinar dos tipos
11 type AdminUser = User & {
12   permissions: string[];
13   lastLogin: Date;
14 };
15
16 // Variantes de UI
17 type ButtonVariant = 'primary' | 'secondary';
18 type ButtonSize = 'sm' | 'md' | 'lg';
```

`interface` → describe la forma de objetos, props de componentes, modelos de API. `type` → unions, estados finitos, alias y transformaciones.

¿Cuándo usar interface vs type?

Guía práctica para frontend



interface — modelos de datos

```
1 // Props de un componente
2 interface ButtonProps {
3   label: string;
4   variant: 'primary' | 'secondary';
5   onClick: () => void;
6   disabled?: boolean;
7 }
8
9 // Modelo de la API
10 interface Product {
11   id: string;
12   name: string;
13   price: number;
14   stock: number;
15 }
16
17 // Extender — reutilizar sin repetir
18 interface FeaturedProduct extends Product {
19   badge: string;
20   discount: number;
21 }
```



type — unions y transformaciones

```
1 // Estados de UI (union literal)
2 type LoadingState = 'idle' | 'loading' | 'error';
3 type Theme = 'light' | 'dark';
4
5 // Respuesta de API (union discriminada)
6 type ApiResponse =
7   | { status: 'ok'; data: Product[] }
8   | { status: 'error'; message: string };
9
10 // Derivar tipos de uno existente
11 type ProductPreview = Pick<Product,
12   'id' | 'name' | 'price'>;
13 type NewProduct = Omit<Product, 'id'>;
14 type ProductDraft = Partial<Product>;
```


Generics — El problema

Sin generics — hay que duplicar para cada tipo

```
1 function firstString(items: string[]): string { return items[0]; }
2 function firstNumber(items: number[]): number { return items[0]; }
3 function firstProduct(items: Product[]): Product { return items[0]; }
4 // → código duplicado ❌
```

Con generics — una sola definición, cualquier tipo

```
1 // <T> es un "placeholder" de tipo que se resuelve al llamar la función
2 function first<T>(items: T[]): T {
3   return items[0];
4 }
5
6 first(['a', 'b', 'c']); // TS infiere T = string → retorna string
7 first([1, 2, 3]);      // TS infiere T = number → retorna number
8 first<Product>(products); // explícito → retorna Product
```

```
1 // Genérico con constraint — T debe tener al menos un `id`
2 function findById<T extends { id: string }>(items: T[], id: string): T | undefined {
3   return items.find(item => item.id === id);
4 }
5
6 findById(products, '1'); // ✅ Product tiene id
7 findById([1, 2, 3], '1'); // ❌ number no tiene id → error en build-time
```


Generics — Utility Types del lenguaje

```
1 interface Product {
2   id: string; name: string; price: number;
3   stock: number; description: string; imageUrl: string;
4 }
5
6 // Partial<T> — todos los campos se vuelven opcionales
7 type ProductDraft = Partial<Product>;
8 const draft: ProductDraft = { name: 'Borrador' }; // ✅ no pide id ni price
9
10 // Pick<T, K> — solo los campos que necesitas
11 type ProductCard = Pick<Product, 'id' | 'name' | 'price' | 'imageUrl'>;
12
13 // Omit<T, K> — todo menos los campos que excluís
14 type CreateProductInput = Omit<Product, 'id'>;
15
16 // Record<K, V> — diccionario tipado
17 type StatusMessages = Record<'idle' | 'loading' | 'success' | 'error', string>;
18 const messages: StatusMessages = {
19   idle: 'Esperando ... ', loading: 'Cargando ... ',
20   success: '¡Listo!', error: 'Algo salió mal.',
21   // Si te olvidás una clave → TS te avisa ❌
22 };
```



Estos utility types son la base del tipado en apps reales: formularios, API calls, componentes reutilizables.

Ejercicio 2

Tipar desde Cero

Estilo Total TypeScript — problema → solución

Ejercicio 2 – Tipar desde cero

⚠ Este código tiene 5 errores de TypeScript. Tu tarea es arreglarlos agregando tipos.

```
1 // 1. `id` puede ser number o string – el tipo no lo refleja
2 let userId: number = 'u-42';
3
4 // 2. Esta función acepta cualquier cosa – anotá sus parámetros y retorno
5 function calculateTotal(items, taxRate) {
6   return items.reduce((sum, item) => sum + item.price * item.quantity, 0) * (1 + taxRate);
7 }
8
9 // 3. El status solo puede ser uno de estos valores – usá un union type
10 let orderStatus = 'shipped'; // luego se asigna 'en_camino' ← inválido
11
12 // 4. Esta interface le falta el campo `stock` que se usa más abajo
13 interface Product { id: string; name: string; price: number; }
14 const available = products.filter(p => p.stock > 0); // ✗ stock no existe
15
16 // 5. Anotá el genérico que le falta a esta función
17 function getLastItem(items) {
18   return items[items.length - 1]; // retorna `any`
19 }
```

Ejercicio 2 — Solución

```
// 1. Union type
let userId: number | string = 'u-42';
console.log('userId:', userId);

// 2. Anotar parámetros y retorno
interface CartItem { price: number; quantity: number; }
function calculateTotal(items: CartItem[], taxRate: number): number {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0) * (1 + taxRate);
}
console.log('Total:', calculateTotal([{"price": 100, "quantity": 2}], 0.19));

// 3. Union type para el status
type OrderStatus = 'pending' | 'processing' | 'shipped' | 'delivered' | 'cancelled';
let orderStatus: OrderStatus = 'shipped';
console.log('Status:', orderStatus);

// 4. Interface con stock
interface Product { id: string; name: string; price: number; stock: number; }
const products: Product[] = [
  { id: '1', name: 'Teclado', price: 29990, stock: 5 },
  { id: '2', name: 'Monitor', price: 89990, stock: 0 },
];
console.log('Disponibles:', products.filter(p => p.stock > 0).map(p => p.name));

// 5. Generic
function getLastItem<T>(items: T[]): T { return items[items.length - 1]; }
console.log('Último:', getLastItem([1, 2, 3]));
```

Resumen — Cheatsheet

JavaScript & DOM

Concepto	Cuándo usarlo
<code>querySelector</code>	Seleccionar un elemento
<code>querySelectorAll</code>	Seleccionar todos los matches
<code>textContent</code>	Modificar texto (seguro)
<code>innerHTML</code>	Renderizar HTML dinámico
<code>classList</code>	Agregar/quitar clases CSS
<code>addEventListener</code>	Escuchar interacciones
Event delegation	Listas dinámicas
Destructuring / Spread	Extraer y copiar datos

TypeScript Basics

Concepto	Cuándo usarlo
<code>: string / number</code>	Parámetros de función
<code>'a' 'b'</code>	Variantes, estados
<code>interface</code>	Forma de objetos y entidades
<code>type</code>	Unions, aliases, intersections
<code>? en campos</code>	Campos opcionales
<code>Partial<T></code>	Formularios de edición
<code>Pick / Omit</code>	Sub-tipos de un modelo
<code><T> generic</code>	Funciones reutilizables

¿Qué sigue? — Recursos recomendados



Total TypeScript

totaltypescript.com

Ejercicios interactivos en el browser, sin instalar nada.

Matt Pocock — ex-Vercel — referencia en la industria



TypeScript Handbook

typescriptlang.org/docs

Documentación oficial. Sección "The Basics" cubre todo lo visto hoy. Útil como referencia cuando tenés una duda específica.

Gratis — mantenido por Microsoft



Próxima clase:

Martes 24 — Web Design Basics

Jueves 26 — Prototyping

¿Preguntas?

Clase 4 – JavaScript DOM & TypeScript Basics



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IIC2182 – Primer Semestre 2026